

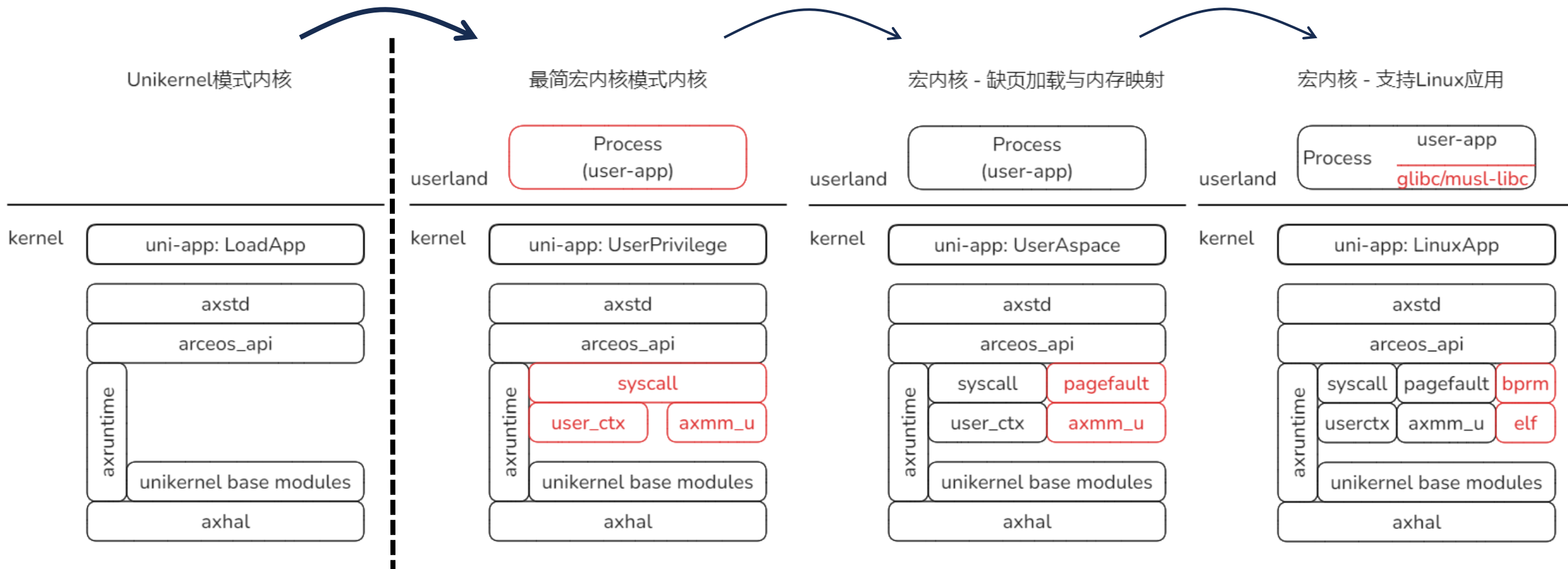
秋冬季训练营三阶段 组件化内核设计与实践(4)

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

2024.11.18

第二部分 Monolithic Kernel宏内核

跨越模式边界



第二周的课程安排

主要内容：基于组件化方法构建宏内核模式。

第一次：从Unikernel到宏内核

以构建最小化的宏内核为目标，说明：

宏内核特点、与Unikernel的差异分析、框架和组件构成、具体实现示例。

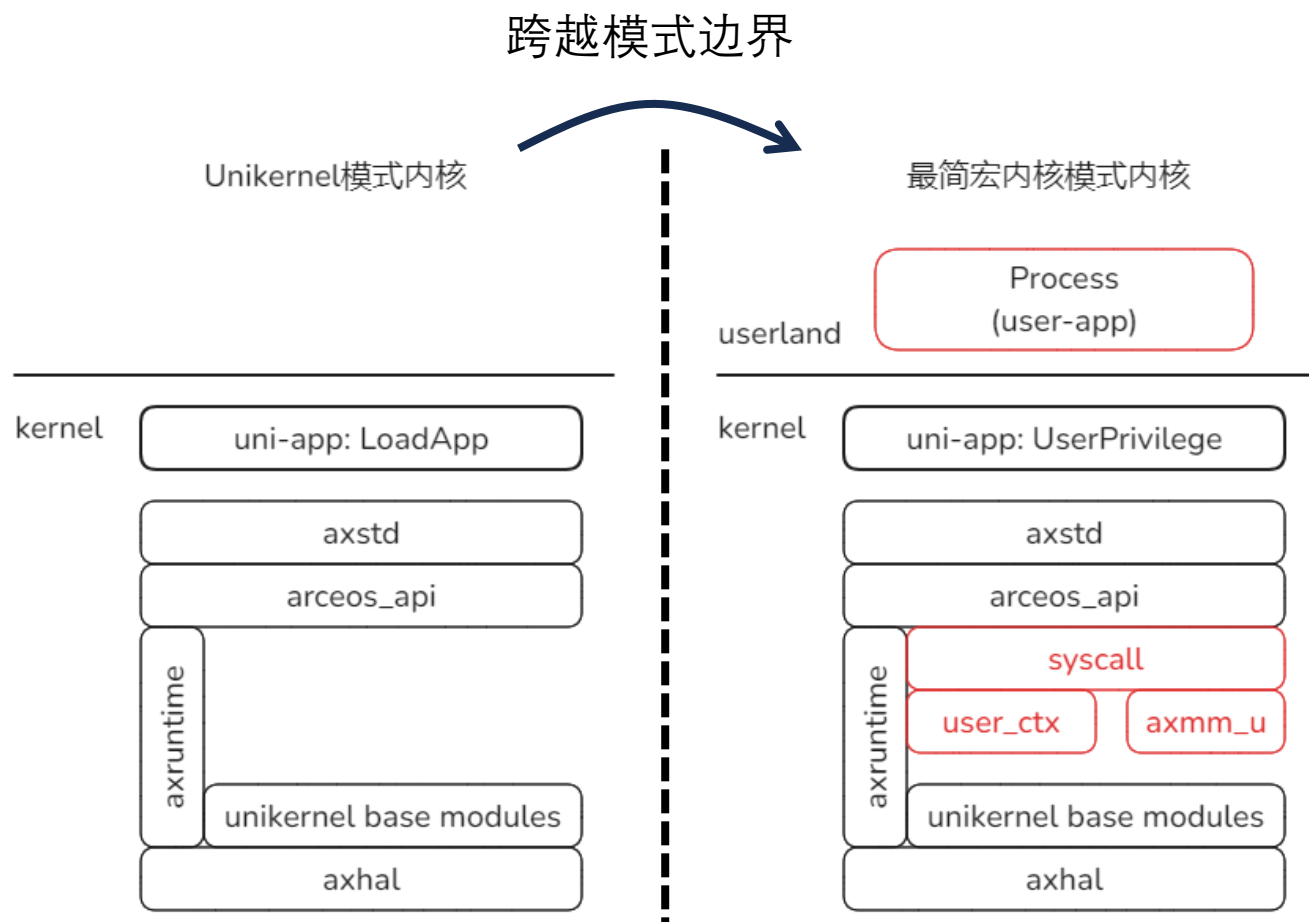
第二次：地址空间管理和支持Linux应用

1. 用户地址空间映射、缺页加载机制；
2. 支持运行最简单的Linux原始应用

第三次：宏内核的开发实践经验 (by 郑友捷老师)

目前宏内核方向的课程基础Starry，主要是由郑老师负责开发的，他也是第四阶段 - 宏内核方向的实习指导。

M.1.0 UserPrivilege



实验命令行:

make payload

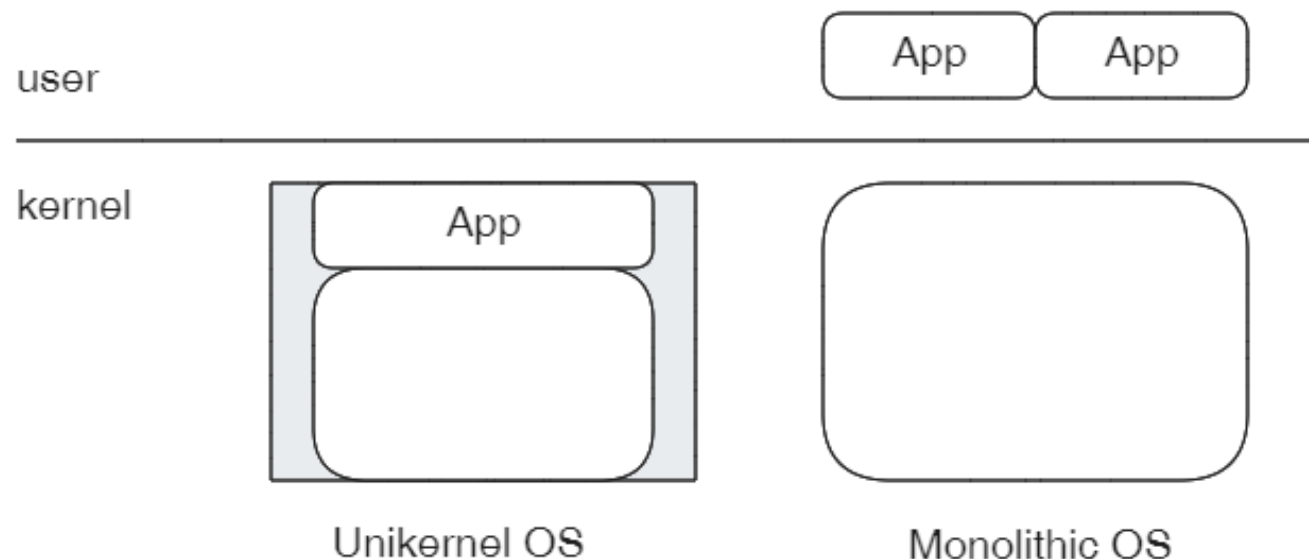
./update_disk.sh ./payload/origin/origin

make run A=tour/m_1_0 BLK=y

本节目标:

1. 从Unikernel扩展为宏内核的基本条件: 特权级、地址空间、系统调用等
2. 用户特权级和特权级上下文切换机制

宏内核模式相对Unikernel模式的特点



Unikernel模式的应用与内核:

- (1) 处于同一特权级 - 内核特权级
- (2) 共享同一地址空间 - 相互可见
- (3) 编译形成一个Image, 一体运行
- (4) Unikernel既是应用又是内核, 二者合体

相对于Unikernel, 宏内核的特点:

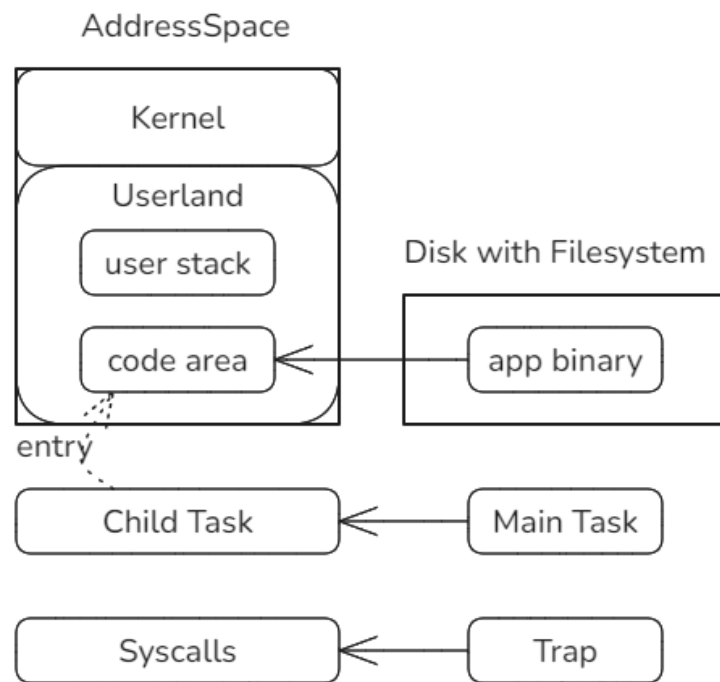
- (1) 增加一个权限较低的用户特权级来运行应用。
- (2) 为应用创建独立的**用户地址空间**, 与内核隔离。
- (3) 内核运行时可以随时加载**应用Image**投入运行。
- (4) 应用与内核界限分明。

需要解决的问题

如何以Unikernel为基础，构建最小化宏内核？

1. 能够创建和管理内核地址空间，为用户地址空间保留低端内存区域
2. 可以从基于块设备的文件系统中搜索和读入应用程序文件
3. 能够创建子线程任务，与主线程并发运行单独的逻辑代码
4. 能够响应异常和中断

当前基础：Unikernel模式

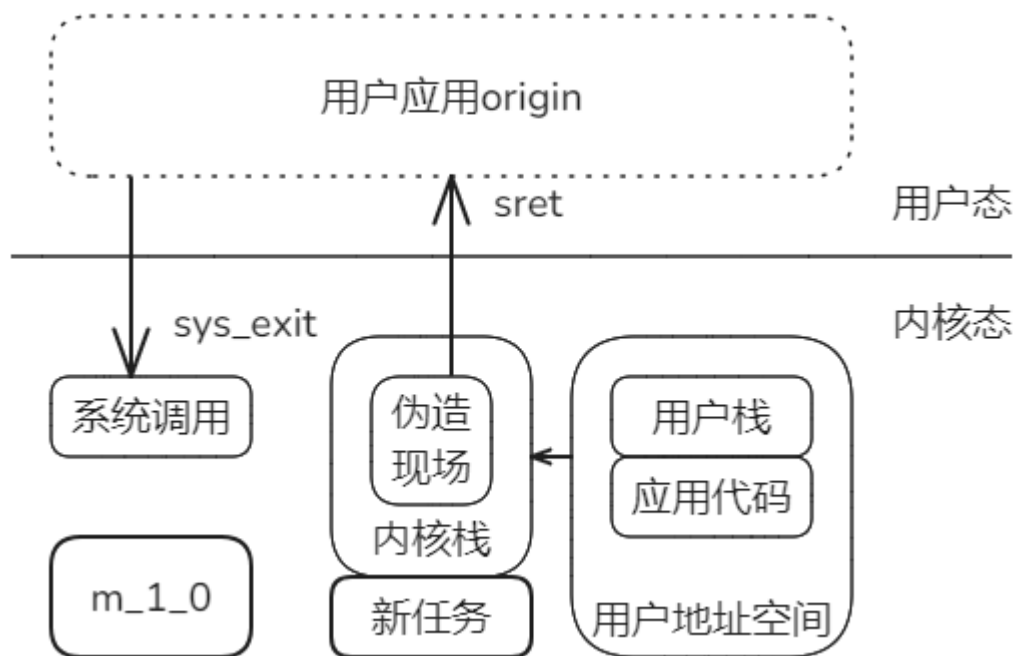


目标：最小化宏内核模式

分析从Unikernel基础到目标最小化宏内核需要完成的增量工作：

1. 用户地址空间的创建和区域映射
2. 在异常中断响应的基础上增加系统调用
3. 复用Unikernel原来的调度机制，针对宏内核扩展Task属性
4. 在内核与用户两个特权级之间的切换机制

结合本节示例m_1_0说明实现过程



示例m_1_0的执行逻辑:

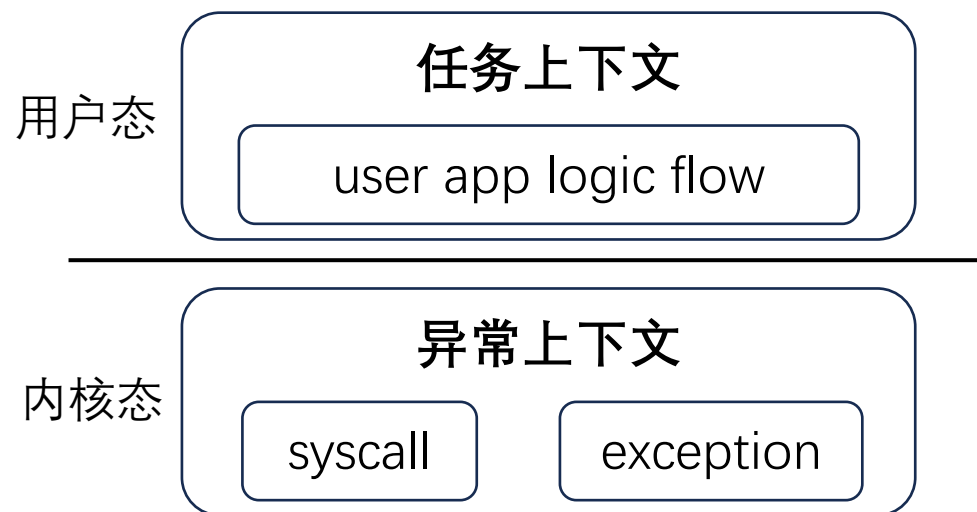
1. 创建用户地址空间
2. 加载应用origin到地址空间
3. 在地址空间中建立用户栈
4. 伪造一个返回应用的环境上下文现场
5. 把伪造现场设置到到新任务的内核栈上
6. 启动新任务执行sret指令返回到用户态, 从应用origin的entry开始执行
7. 应用origin只包含一行代码, 即执行系统调用sys_exit
8. 注册在异常中断向量表中的系统调用响应函数处理sys_exit, 内核退出

结合本节示例m_1_0说明实现过程

从前面的分析可见，宏内核模式为用户应用建立了两类上下文。

用户应用进程在它们之间交替运行：

1. 任务上下文 - 用户态：正常执行应用逻辑，也称为进程上下文
2. 异常上下文 - 内核态：处理系统调用与异常 *中断上下文与异常上下文类似，这里不讨论*



以下重点从这两种上下文角度，说明m_1_0最小化宏内核的实现过程。

本节示例m_1_0的启动过程概览

tour/m_1_0/src/main.rs

```
fn main() {  
    // A new address space for user app.  
    let mut uspace = axmm::new_user_aspace().unwrap();  
  
    // Load user app binary file into address space.  
    if let Err(e) = load_user_app("/sbin/origin.bin", &mut uspace) {  
        panic!("Cannot load app! {:?})", e);  
    }  
  
    // Init user stack.  
    let ustack_top = init_user_stack(&mut uspace, true).unwrap();  
    ax_println!("New user address space: {:#x?}", uspace);  
  
    // Let's kick off the user process.  
    let user_task = task::spawn_user_task(  
        Arc::new(Mutex::new(uspace)),  
        UspaceContext::new(APP_ENTRY.into(), ustack_top),  
    );  
  
    // Wait for user process to exit ...  
    let exit_code = user_task.join();  
    ax_println!("monolithic kernel exit [{:?}] normally!", exit_code);  
}
```

1. 为应用创建独立的用户地址空间
涉及组件: axmm

2. 加载应用程序代码到地址空间
涉及组件: axfs, axmm

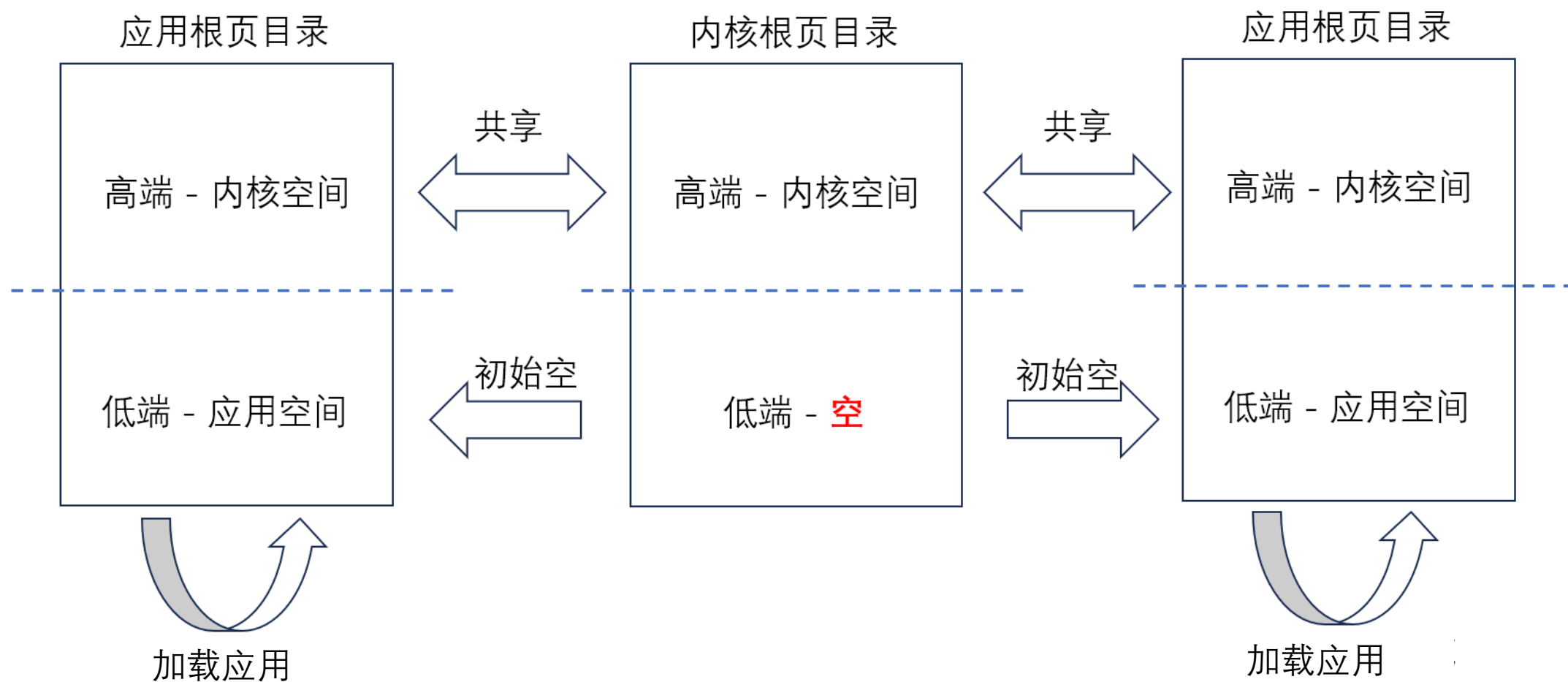
3. 初始化用户栈
涉及组件: axmm

4. 创建用户任务
涉及组件: axtask (with taskext)

5. 让出CPU, 使得用户任务运行
涉及组件: axtask, axhal

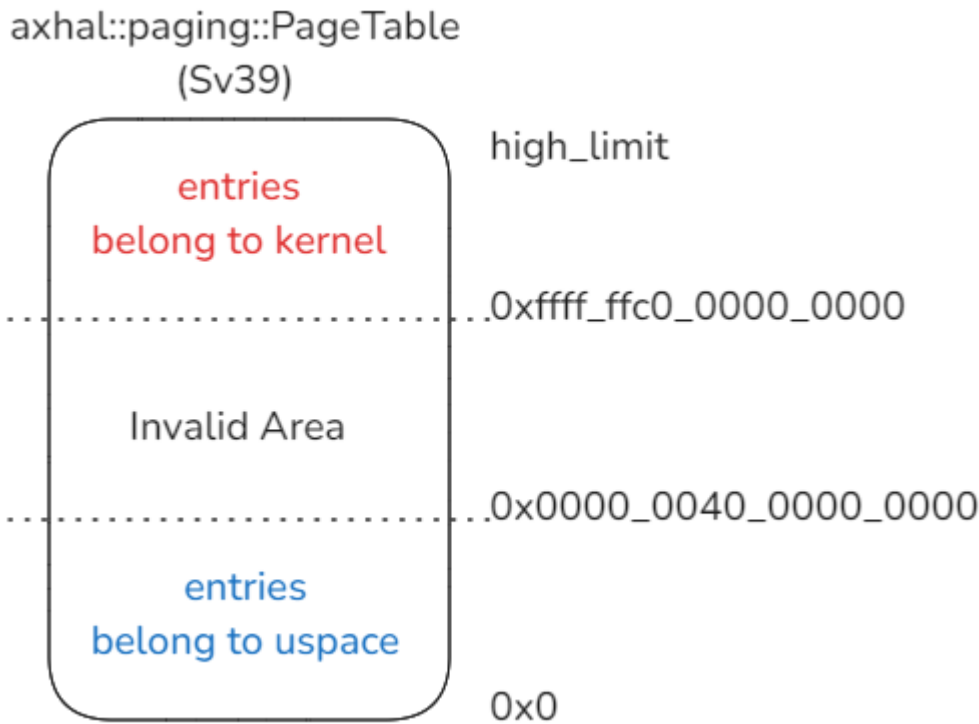
用户空间 - 地址空间管理与复制原理

页表分为高低两个部分：高端作为内核空间，低端作为用户应用空间。
以初始的内核根页表为模板，为每个应用进程复制独立页表。内核空间共享，用户空间独立使用。



用户空间 - 概念与具体实现对应关系

页表用数据结构 PageTable实现； 地址空间用数据结构AddrSpace实现。

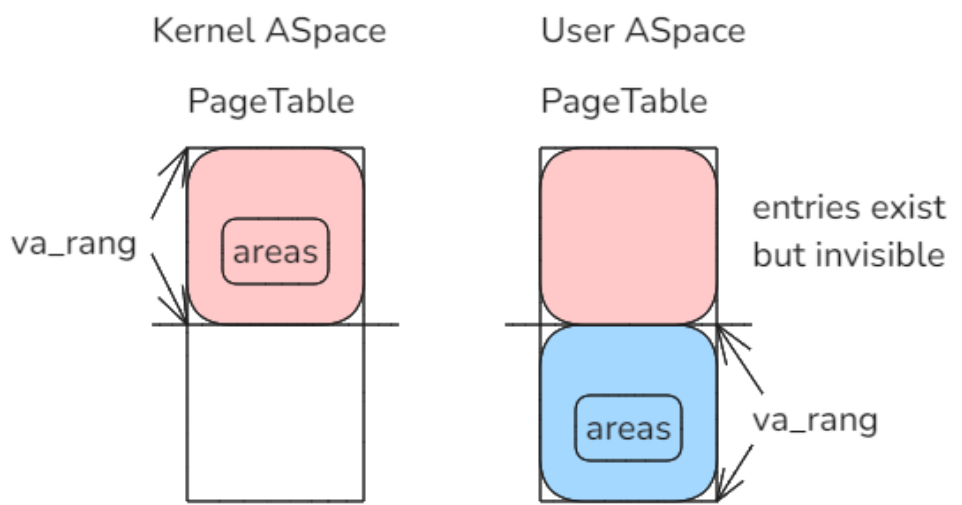


高端有效范围留给内核
低端有效范围留给应用

```
/// The virtual memory address space.
pub struct AddrSpace {
    va_range: VirtAddrRange,
    areas: MemorySet<Backend>,
    pt: PageTable,
}
```

可见范围
区域列表
拥有的页表

内核地址空间与用户地址空间包含PageTable差别



用户应用的编译链接

用户应用构建方式： Rust工具链 + Rust嵌入式汇编

示例： payload/origin

```
#[no_mangle]
unsafe extern "C" fn _start() -> ! {
    core::arch::asm!(
        "li a7, 93",
        "ecall",
        options(noreturn)
    )
}
```

93是 Sys_exit编号
通过ecall触发系统调用

首先编译origin生成ELF格式， 然后被工具链转化为二进程BIN格式。

```
cargo build -p origin --target riscv64gc-unknown-none-elf --release
```

```
rust-objcopy --binary-architecture=riscv64 --strip-all -O binary [origin_elf] [origin_bin]
```

BIN格式作为exercises/m_1_0使用的用户应用image。

把用户应用安装到根文件系统

通过命令行make disk_img已经创建磁盘设备disk.img，并建立文件系统(fat32)。安装用户应用就是mount该磁盘设备文件到 ./mnt目录，然后更新应用程序image。

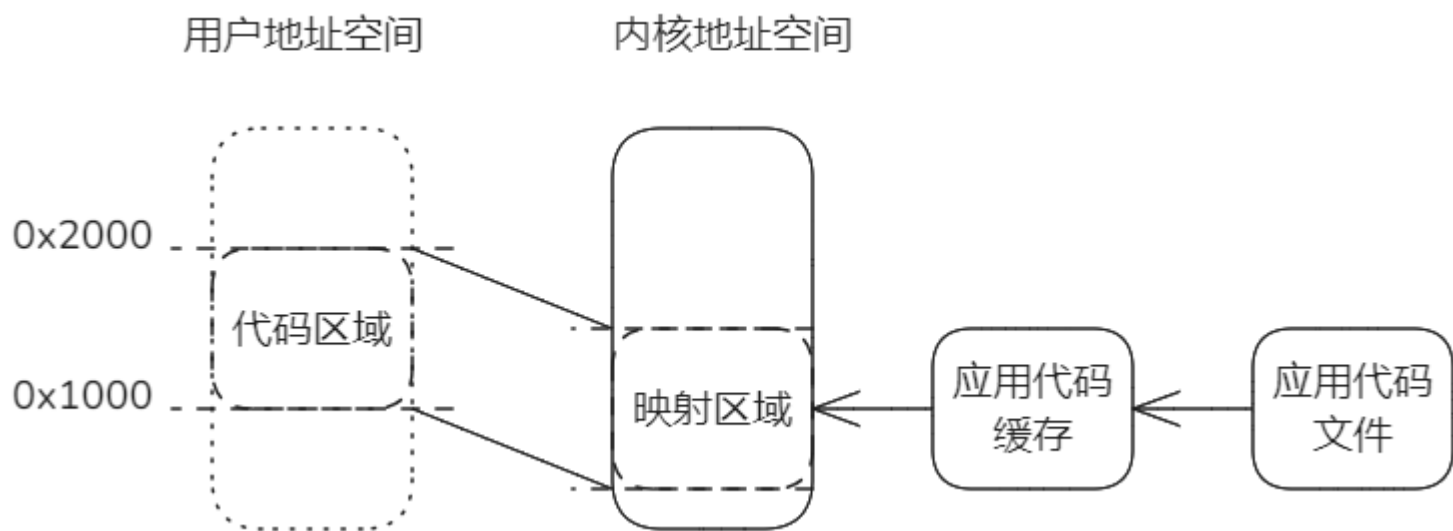
例如，安装应用origin的二进制Image：

```
mkdir -p ./mnt  
mount $(1) ./mnt  
mkdir -p ./mnt/sbin  
cp /tmp/origin.bin ./mnt/sbin  
umount ./mnt
```

把应用加载到用户地址空间

第一步，从文件加载代码到内存缓冲区。

第二步，为用户地址空间代码区域建立映射，拷贝代码到被映射页面中。



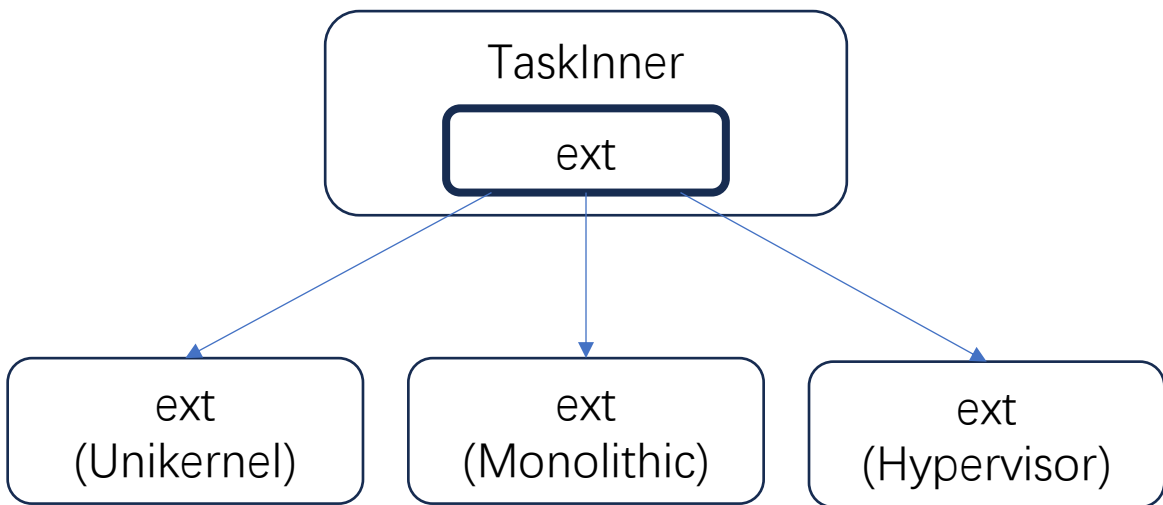
应用尚未运行

内核代替应用完成地址空间映射和加载代码

创建用户任务 - 任务扩展属性

TaskInner的扩展:

在其中增加一个ext的结构体成员,
包装与宏内核相关的各个成员。
对原始Unikernel是空结构。



```
/// Task extended data for the monolithic kernel.
pub struct TaskExt {
    /// The process ID.
    pub proc_id: usize,
    /// The user space context.
    pub uctx: UspaceContext,
    /// The virtual memory address space.
    pub aspace: Arc<Mutex<AddrSpace>>,
}

impl TaskExt {
    pub const fn new(uctx: UspaceContext, aspace:
        Self {
            proc_id: 1,
            uctx,
            aspace,
        }
    }
}

axtask::def_task_ext!(TaskExt);
```

通过宏def_task_ext为宏内核扩展属性

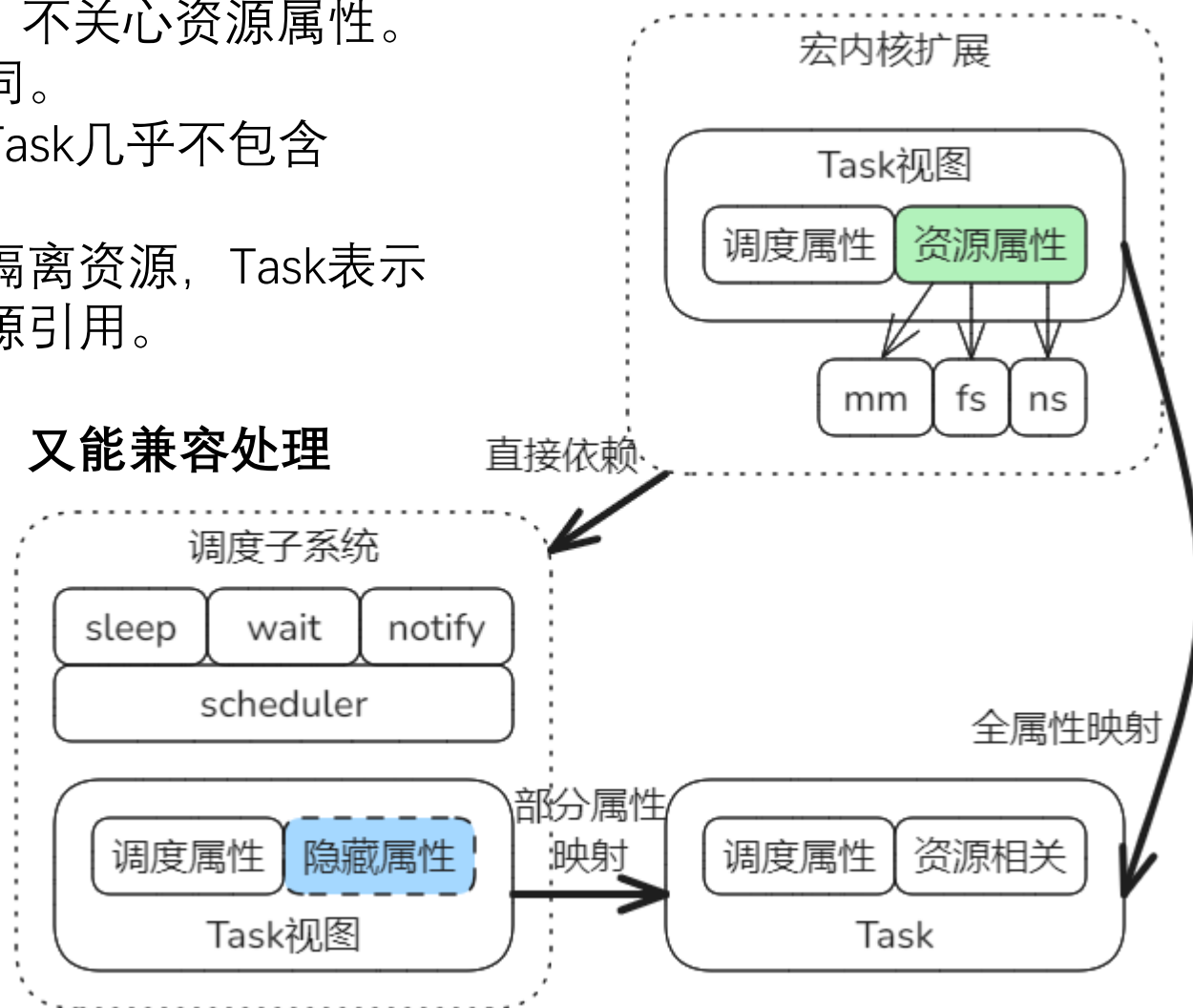
任务属性扩展机制的目的

对于各类内核模式，调度子系统机制是基本一致的，调度仅关心Task中与调度相关的属性，不关心资源属性。模式之间区别主要就在于资源属性不同。

Unikernel模式下，资源都是全局的，Task几乎不包含资源属性；

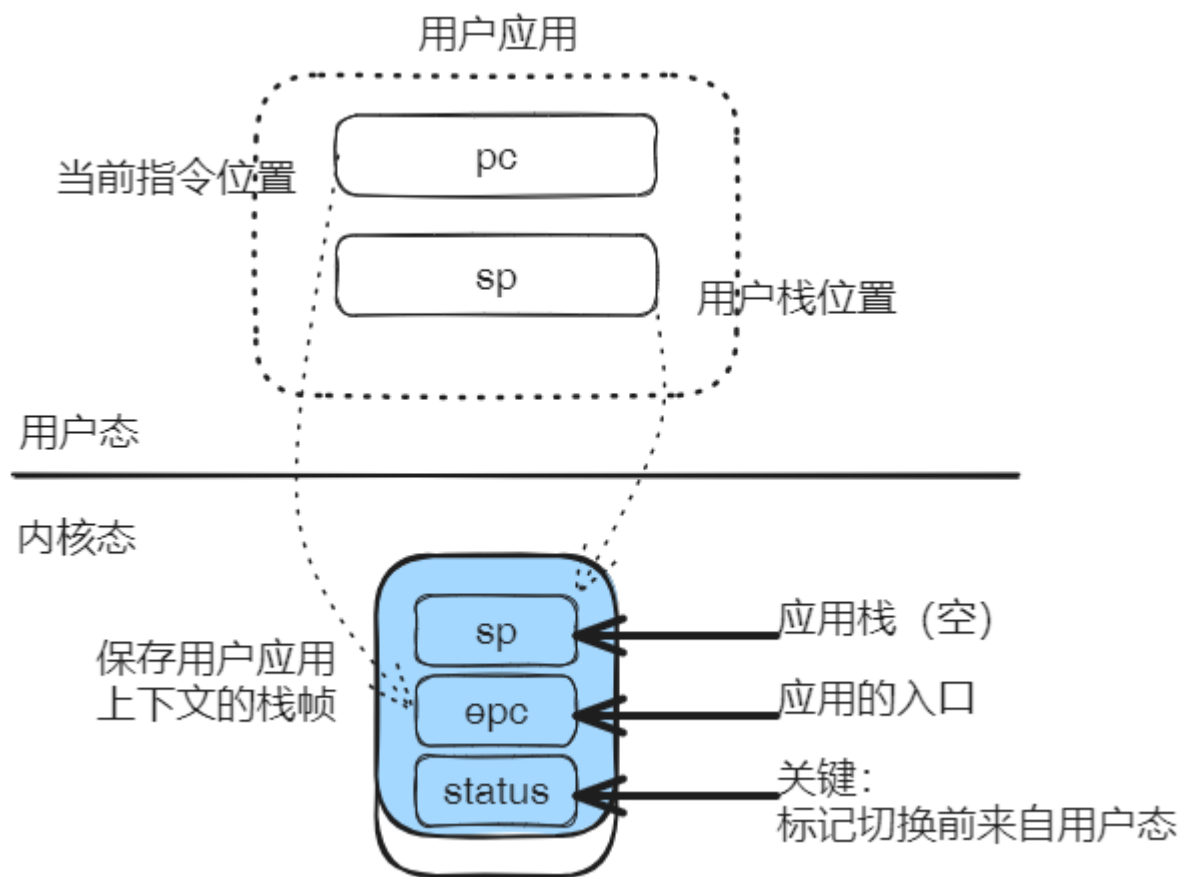
宏内核模式下，以进程为单位管理和隔离资源，Task表示进程时，其中就要包含属于自己的资源引用。

目的：尽可能复用共性的调度子系统，又能兼容处理各种模式的个性部分 - 资源管理。



进入用户态 - 让出CPU, Kick off 用户任务

对于Riscv等多数体系结构来说，并不存在一个专门指令实现从内核态到用户态的切换。
解决方法：在内核态伪造一个异常上下文现场，假装来自用户态，然后用sret指令返回去。



```
pub unsafe fn enter_uspace(&self, kstack_top: VirtAddr)
    use riscv::register::{sepc, sscratch};

    super::disable_irqs();
    sscratch::write(kstack_top.as_usize());
    sepc::write(self.0.sepc);
    // Address of the top of the kernel stack after sav
    let kernel_trap_addr = kstack_top.as_usize() - core
    asm!(
        mv      sp, {tf}
        STR     gp, {kernel_trap_addr}, 2
        LDR     gp, sp, 2

        STR     tp, {kernel_trap_addr}, 3
        LDR     tp, sp, 3

        LDR     t0, sp, 32
        csrw    sstatus, t0
        POP_GENERAL_REGS
        LDR     sp, sp, 1
        sret",
        tf = in(reg) &(self.0),
        kernel_trap_addr = in(reg) kernel_trap_addr,
```

示例m_1_0对系统调用的响应

modules/axhal/src/arch/riscv/trap.rs

```
fn riscv_trap_handler(tf: &mut TrapFrame, from_user: bool) {  
    let scause = scause::read();  
    match scause.cause() {  
        #[cfg(feature = "uspace")]  
        Trap::Exception(E::UserEnvCall) => {  
            tf.regs.a0 = crate::trap::handle_syscall(tf, tf.  
            tf.sepc += 4;  
        }  
    }  
}
```

1. 扩展异常响应入口函数
对应处理系统调用

tour/m_1_0/src/syscall.rs

```
#[register_trap_handler(SYSCALL)]  
fn handle_syscall(tf: &TrapFrame, syscall_num: usize) -> isize {  
    ax_println!("handle_syscall ...");  
    let ret = match syscall_num {  
        SYS_EXIT => {  
            ax_println!("[SYS_EXIT]: system is exiting ..");  
            axtask::exit(tf.arg0() as _)  
        },  
        _ => {  
            ax_println!("Unimplemented syscall: {}", syscall_num);  
            -LinuxError::ENOSYS.code() as _  
        }  
    };  
    ret  
}
```

2. 具体的系统调用实现
当前示例仅实现了SYS_EXIT

课后练习

[page_fault]: 处理缺页异常。

预备：把tour/m_1_0/src/main.rs的第39行，init_user_stack的第二个参数从true改为false。

```
// Init user stack.  
let ustack_top = init_user_stack(&mut uspace, false).unwrap();
```

执行

make payload

./update_disk.sh payload/origin/origin

make run A=tour/m_1_0/ BLK=y

```
Enter user space: entry=0x1000, ustack=0x4000000000, kstack=VA:0xffffffffc080687010  
[ 0.328589 0:4 axhal::arch::riscv::trap:24] No registered handler for trap PAGE_FAULT  
[ 0.331334 0:4 axruntime::lang_items:5] panicked at modules/axhal/src/arch/riscv/trap.rs:25:9:  
Unhandled User Page Fault @ 0x1002, fault_vaddr=VA:0xffffffffc (WRITE | USER):  
TrapFrame {  
  regs: GeneralRegisters {  
    ^  
    ^
```

要求：实现handle_page_fault响应函数，即实现Lazy映射。

提示：

1. handle_page_fault的注册方式参考handle_syscall的注册宏。
2. 答案就是tour/m_2_0，先不看答案，尝试自己补齐实现。